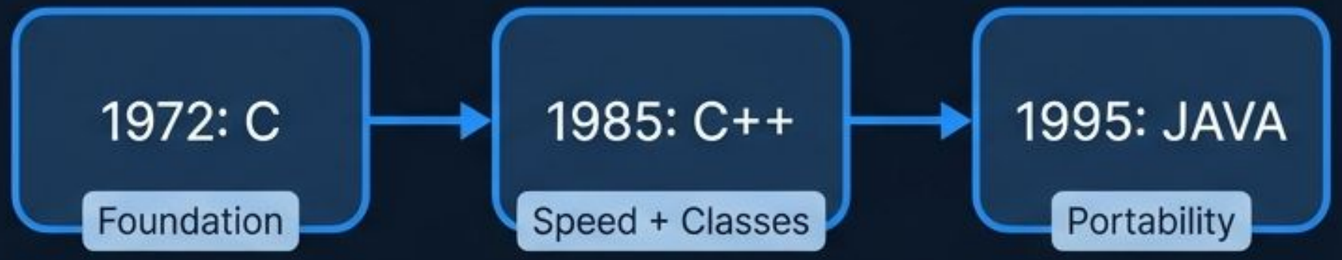
☕ Day 1 — Java Series

The Rebirth of a Legend

FROM C++ TO JAVA: A HISTORY

- **The 1980s & 90s context**
  C and C++ ruled the programming world
- **The strengths of the era**
  Extremely fast execution speed, low-level hardware access
- **The emerging challenge**
  Rise of the Internet and consumer electronics created a need for a “General Purpose” language

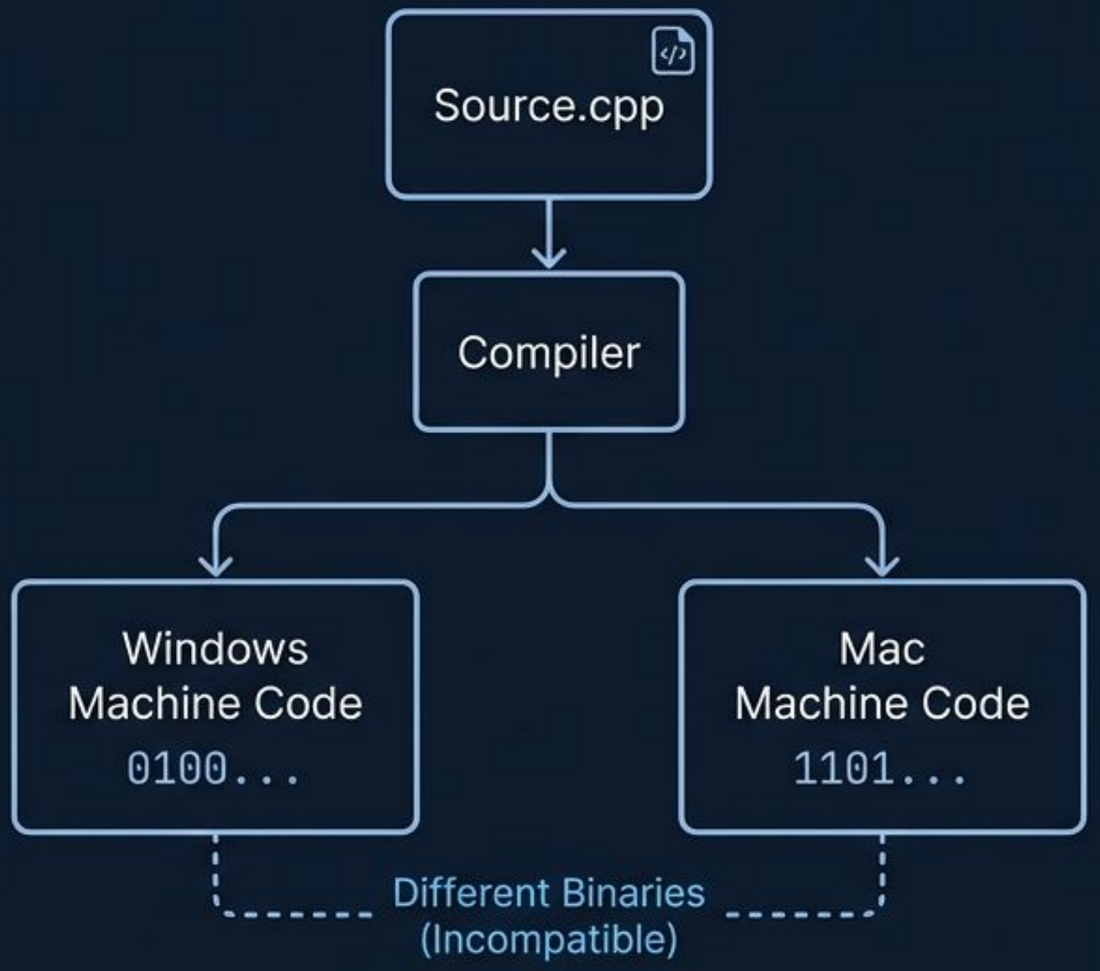


☕ Day 1 — Java Series

The C/C++ Problem

UNDERSTANDING PLATFORM DEPENDENCY

- **Defining a “Platform”**
  Combination of OS + Processor, e.g., Windows + Intel
- **The compilation constraint**
  C++ compiles directly to Machine Code specific to the hardware
- **The portability issue**
  Code compiled for Windows fails on Mac; requires recompiling for every target

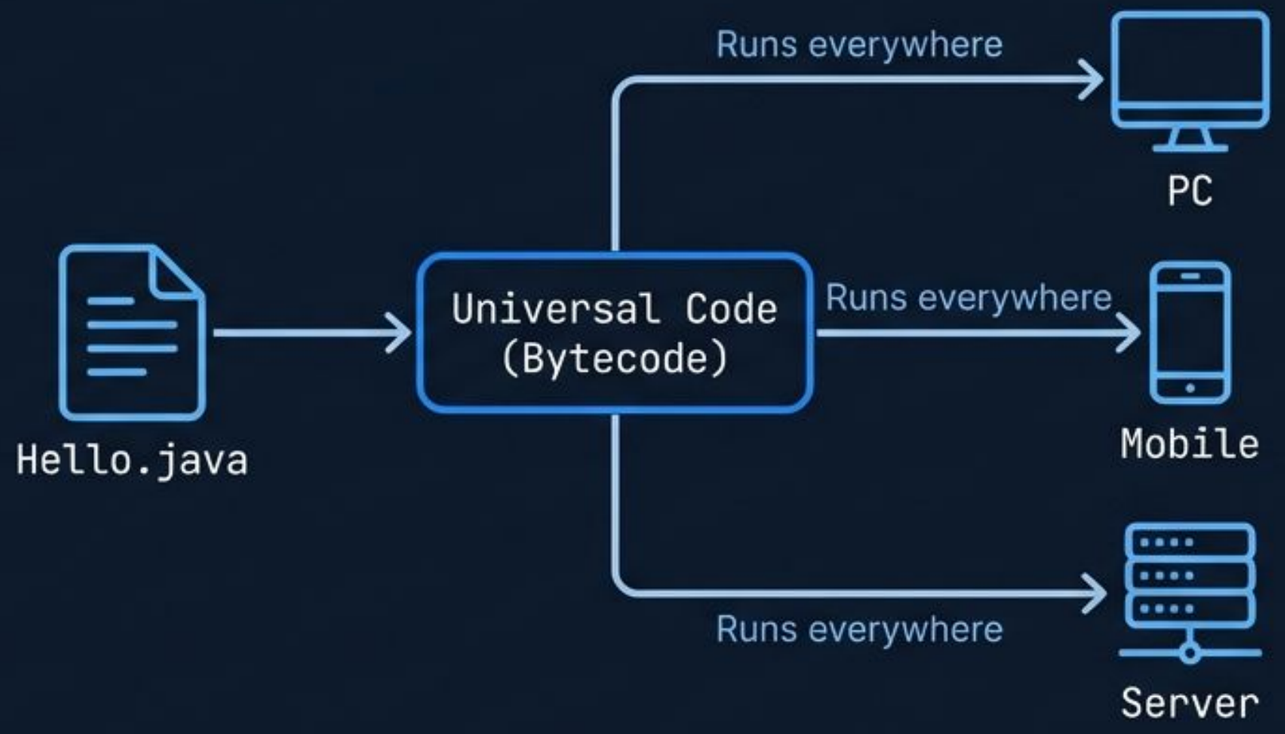


☕ Day 1 — Java Series

Java's Secret Weapon

WRITE ONCE, RUN ANYWHERE (WORA)

- **The paradigm shift**
  Code is written once, not rewritten for every device
- **The "Translator" Model**
  Source code doesn't talk to hardware directly
- **Platform Independence**
  The language is neutral; only the runtime is specific
- **Universal compatibility**
  From servers to set-top boxes

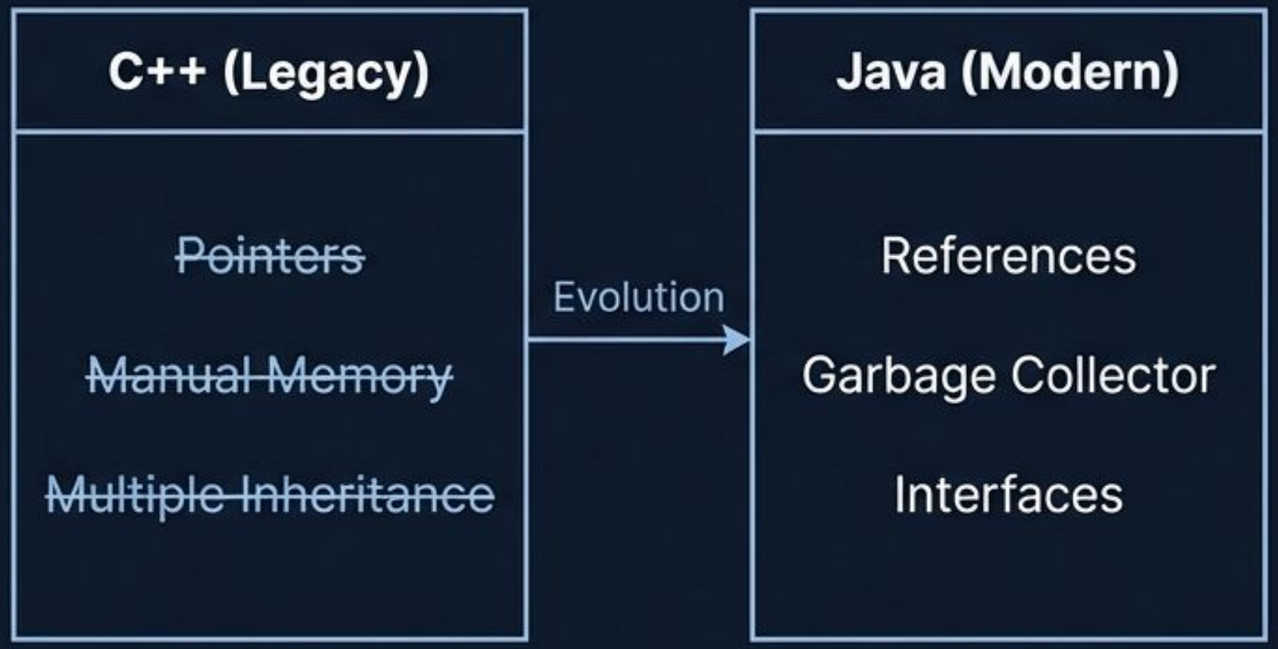


☕ Day 1 — Java Series

Simpler by Design

REMOVING THE COMPLEXITY

- **Removing dangerous features**
  No Pointers to prevent direct memory access; No Operator Overloading
- **Automated Memory Management**
  Garbage Collection handles cleanup automatically; no `malloc` or `free`
- **Simplified Inheritance**
  Eliminated 'Multiple Inheritance' confusion

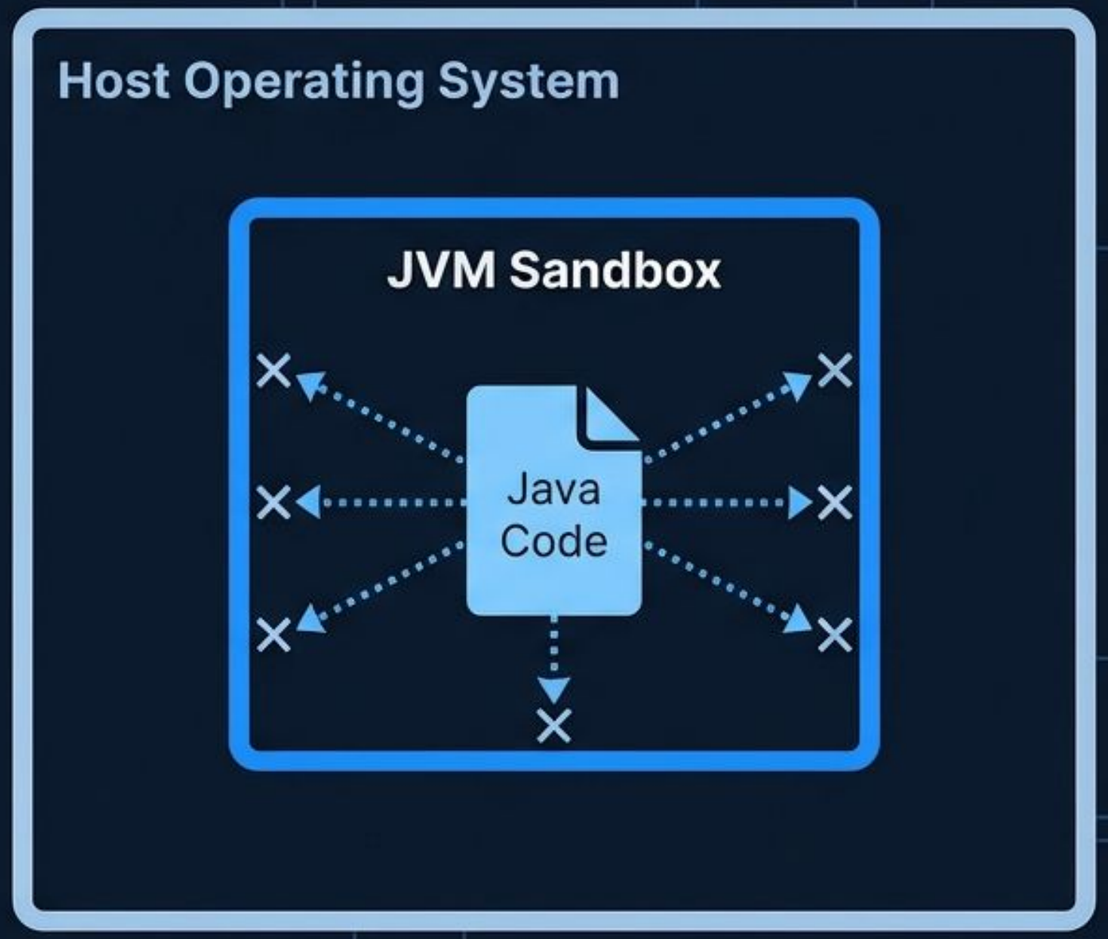


☕ Day 1 — Java Series

The Security Guard

THE SANDBOX MODEL

- **The Early Web Context**
  (Applets allowed code to run in browsers)
- **The Security Risk**
  (Downloading unknown code is dangerous)
- **The Solution: JVM Sandbox**
  (Code runs in a restricted environment with no direct access to Host OS)

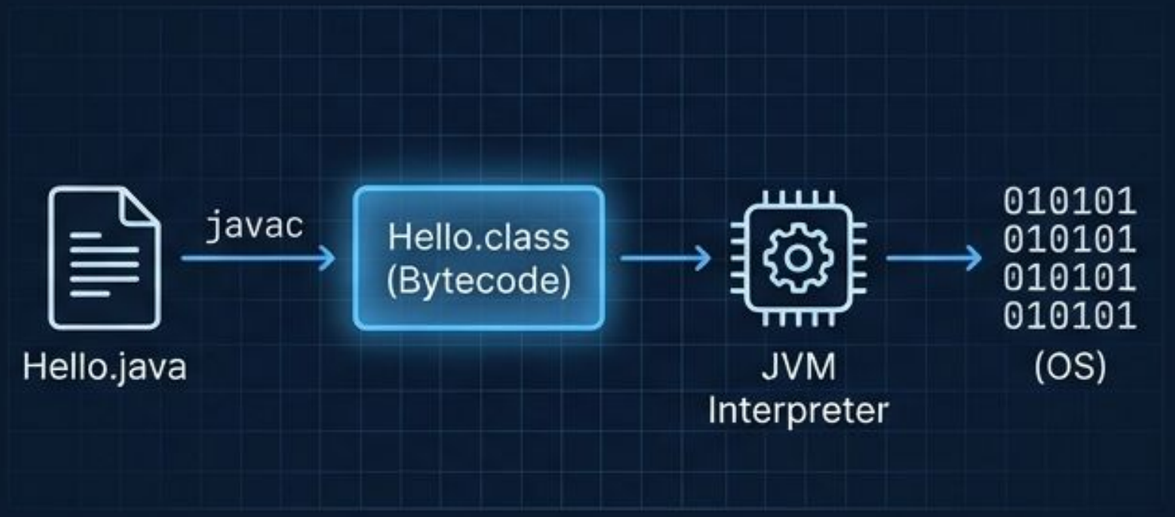


☕ Day 1 — Java Series

How Java Actually Works

SOURCE TO BYTECODE TO MACHINE

- **Step 1: Compilation**
  `javac` converts source code to Bytecode
- **Step 2: The Magic of Bytecode**
  `.class` files are platform-independent Universal Language
- **Step 3: Execution (JVM)**
  JVM reads Bytecode and translates to Machine Code; JVM is platform-dependent

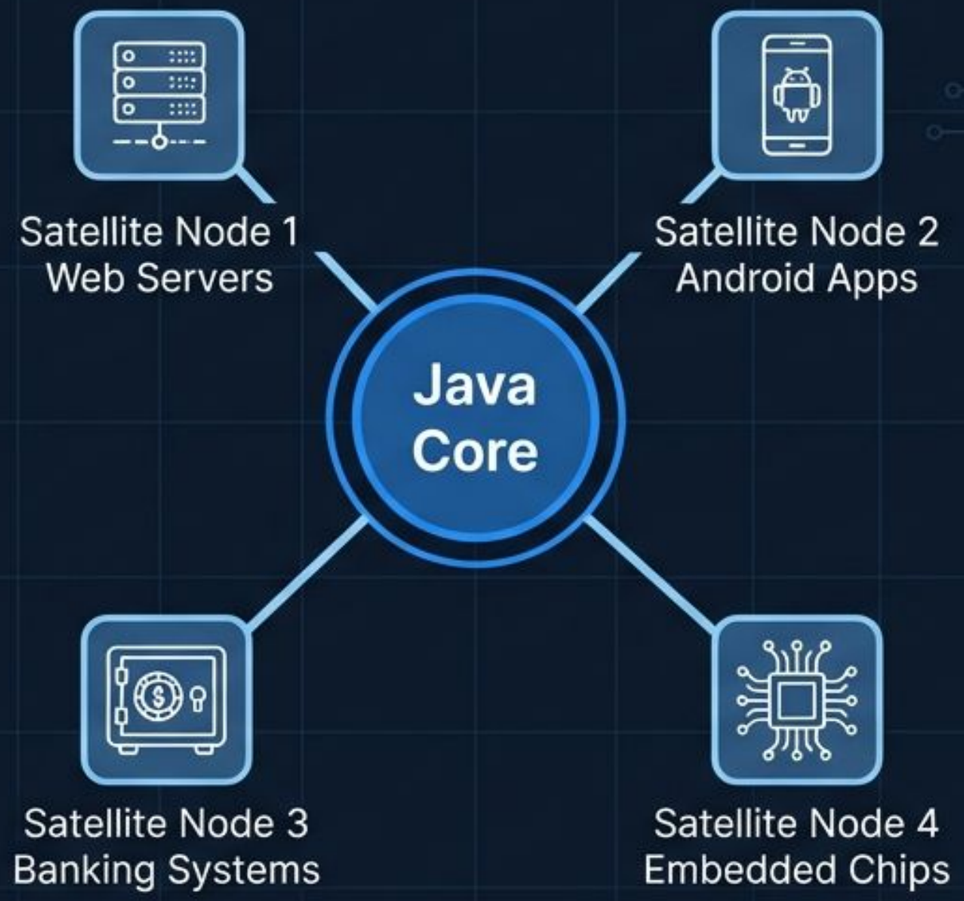


☕ Day 1 — Java Series

Java is Everywhere

GENERAL PURPOSE LANGUAGE

- **Enterprise Backend**
  Servlets and large-scale server applications
- **Legacy Frontend**
  Applets inside browsers - discontinued
- **Mobile & Embedded**
  Android development and Smart Devices
- **Scale**
  Billions of devices run Java today



☕ Day 1 — Java Series

Key Takeaways

FOUNDATIONS LOCKED IN

- **Java vs C++**
  (Platform Independent vs Dependent)
- **The Core Trio**
  Simple: Auto memory management;
  Portable: `WORA` via `Bytecode`;
  Secure: Sandbox execution
- **Architecture**
  Source → `Bytecode` → JVM

Next Step: Day 2

Java Internals & First Program